Assignment 3: 2D UAV Attitude, Translation, and Cascade Control

Student Information

Item	Entry Field
Name	Thomas Rompré
Student ID	B990389904
Submission Date	6/23/2026
MATLAB Version	R2023b Update 3 (23.2.0.2409890)

Instructions

- Run the scenario specified in each part, and write the gains you changed and your observations.
- Paste MATLAB plots or animation screenshots into the figure boxes.
- Keep the discussion short. You may use terms such as stable, slow, oscillatory, and unstable.

Part 1 — Attitude Control Only

Expected content: Control only the attitude angle, and record how gain changes affect convergence, oscillation, and damping.

Setup

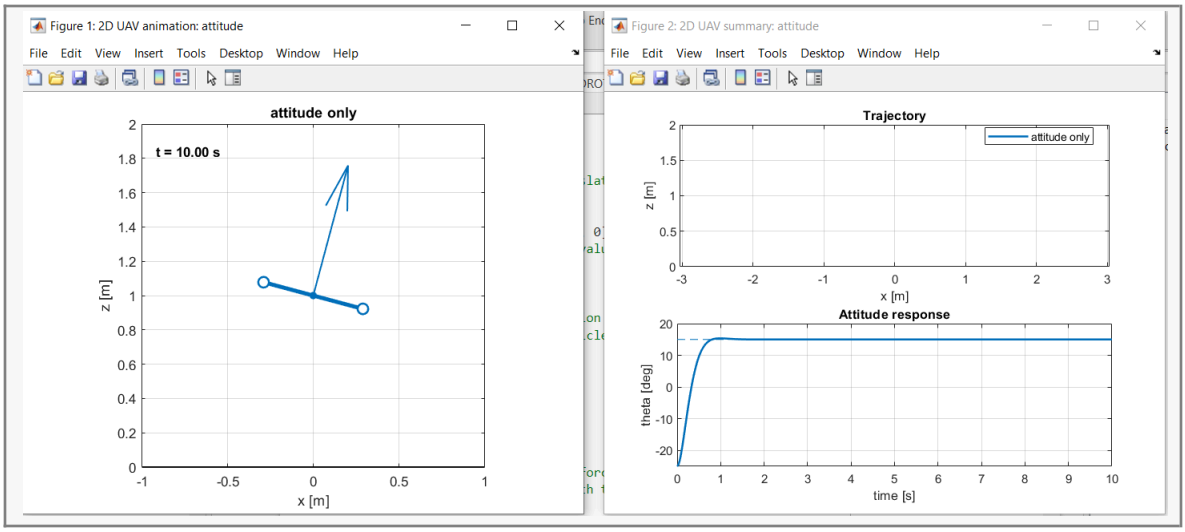
```
scenario     = 'attitude';
compareGains = false;
doAnimation  = true;
doPlots      = true;
```

Modified Values

Case	p.att.kp	p.att.kd	theta0	theta_d	Observation Notes
Nominal Case	0.6	0.18	0	15*pi/180	Takes about 1 second to reach thetaD, or desired angle of 15 degrees
No Derivative Term	0.6	0	0	15*pi/180	Wildly flails between one direction and the other, the amount of turning (or attitude correction) becomes larger with every *swing* to the opposite reaction due to the absence of damping
Weak Attitude Control	0.3	0.09	0	15*pi/180	A less effective version of the nominal case (due to the halving of the damping and attitude control). Took about 2 seconds to stabilize the angle, and overshot the angle after just under a second by roughly 5 degrees before adjusting. In the nominal case, little to no overshooting of the attitude response was

					visible, and the desired attitude was reached almost twice as fast.
Strong Attitude Control	2.0	0.18	0	15*pi/180	Displays stronger overshoot, similarly to the weak attitude control values ironically, but required more stabilization before precisely hitting the desired attitude value.
Best Tuning	1.0	0.25	0	15*pi/180	After some manual fiddling to find out the fastest way to reach a stable thetaD value this is the result i found, which manages to reach theta d within roughly 0.6 seconds.

Figure 1: Attitude response $\theta(t)$ and $\theta_d(t)$, or animation screenshot



Short Discussion

What to write: the smoothest case, the no-derivative case, and the high-proportional-gain case.

Entry field:

The best tuning values I reached after some testing seems to be the ‘’smoothest’’ i can achieve in this simulation. By which we mean the least amount of correction was necessary and the least amount of overshoot occurred in the shortest amount of time possible. Finding that desired balance is, at this level, an unfortunate question of trial and error to find the perfect balance of proportional gain and damping to a specific scenario (as the hardware capabilities of the quadrotor and the environmental conditions we wish to operate it in will inevitably affect the results, requiring a re-calibration of those values). High proportional gain will not necessarily improve correction due to higher values making it much more likely to overshoot, requiring further correction. The complete removal of damping means that the correction values keep increasing exponentially, which increases the error exponentially and then goes back to increasing the correction value, creating a feedback loop where the attitude will spin out of control.

Part 2 — Translation Control Only

Expected content: Use virtual position control while ignoring attitude, and check the roles of position gain and velocity damping.

Setup

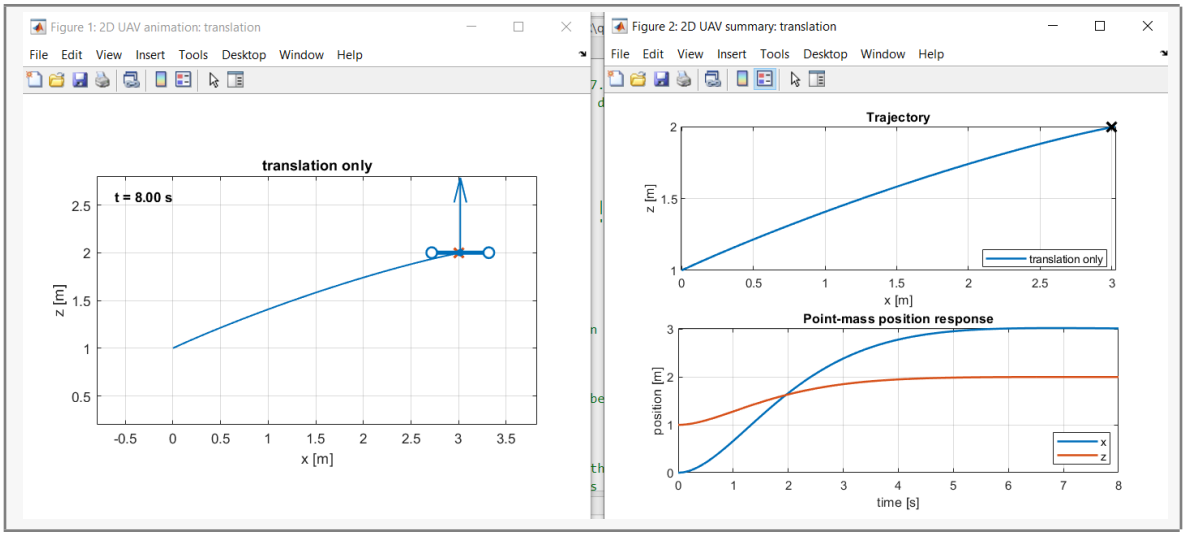
```
scenario     = 'translation';
compareGains = false;
```

Modified Values

Case	kp_x	kp_z	kd_x	kd_z	Target Position	Observation Notes
Nominal Case	0.70	1.00	1.40	1.80	3,1	Target position reached with stability at roughly 5.5 seconds, z value reached first, no overshoot or correction visibly necessary on both axes
No Velocity Damping	0.7	1.00	0	0	3,1	Very high overcorrection causing the quadrotor to constantly overshoot on every axis,

						making it impossible to reach stability by a large margin, similar to attitude control. It does not, however, seem to create a feedback loop which causes the correction / error to increase exponentially over time.
High x-Direction P Gain	2.0	1.0	1.40	1.80	3,1	Takes slightly more time to reach target position and overshoots twice before correcting and reaching target position.
Best Tuning	1.0	1.5	1.775	2.0	3,1	Very *slightly* increases the time required for reaching the target position at stability by roughly a tenth of a second. This is mostly due to the significant increase in damping applied in order to prevent overcorrection from the increased P gain.

Figure 2: Position response x(t), z(t), and target position



Short Discussion

What to write: the roles of p.pos.kp and p.pos.kd, and why this model is insufficient as a real UAV model.

Entry field:

The core problem with this model not being representative of a real-life situation is the attitude not being taken in consideration. The quadrotor's motion is entirely based on a downward ''push'', what direction this downward is, and therefore *where* the push leads is entirely reliant on the current attitude. What this model *does* show us is the effects of proportional gain, which in the simulation seems to behave as the amount of correction being applied to reach a target position and stability, while the damping keeps the effects of proportional gain within a realistic scope in regards to the current position and the target position, this makes stabilization act faster, but makes reaching the target position slightly slower in turn.

Part 3 — Full Cascade Control

Expected content: Compare how the outer loop generates the desired force and how the inner loop tracks the desired attitude.

Setup

```
scenario     = 'cascade';
compareGains = true;
```

Code to Inspect

From uav2d_controller.m, paste the section that converts desired force into thrust T and desired attitude theta_d.

```
% Paste the relevant code here
Line 43 to 45
T = norm(F);
      theta_d = atan2(F(1), F(2));
      theta_d = sat(theta_d, -p.thetaMax, p.thetaMax);
//End Code
This is also dependent on line 41 to determine desired force:
F = p.m*([0; p.g] + a_cmd);
And theta_d is used to calculate tau to be used in the controller later on in line 50:
u.tau = sat(tau, -p.tauMax, p.tauMax);
```

Modified Values

Case	Attitude Gains	Position Gains	Tracking of theta_d and theta	Observation Notes
Nominal Cascade	unmodified	unmodified	unmodified (assuming these values are from att.kd and pos.kd respectively)	Seems to import values from the previous tasks as i did not bother to revert them to defaults (couldn't find it in the assignment PDF if that was the case). Seems to reach the desired position and attitude relatively quickly with some expected

				slowdown once near target position. If my assessment is correct, this seems to inherit the best tuning values without us having to manually assign them to P(1) in uav2d_gain_cases.m. As not assigning them gives the same results as if I did (or the assigning is simply not doing anything)
Slow Inner Loop	0.08		0.05, unmodified	Overshoots due to incorrect attitude. Seems like it would require more than 8 seconds to stabilize at target position and attitude (which is bad)
Strong Outer Loop	unmodified	[2.0 ; 1.0]	Unmodified, [1.5 ; 1.8]	As values would suggest the quadrotor seems to move in larger “sweeps” and require significant correction. This is a similar problem as we see in the slow inner loop albeit it seems more pronounced.
Your Improvement	1.0	[1.0 ; 1.5]	0.25 / [1.775; 2.0]	By applying the best tuning values from the last two tasks We seem to quickly and smoothly reach

				the target position and desired attitude, with little to no correction or overshoot required / caused. Fancy that.

Figure 3: Cascade-control trajectory comparison, or animation-comparison screenshot

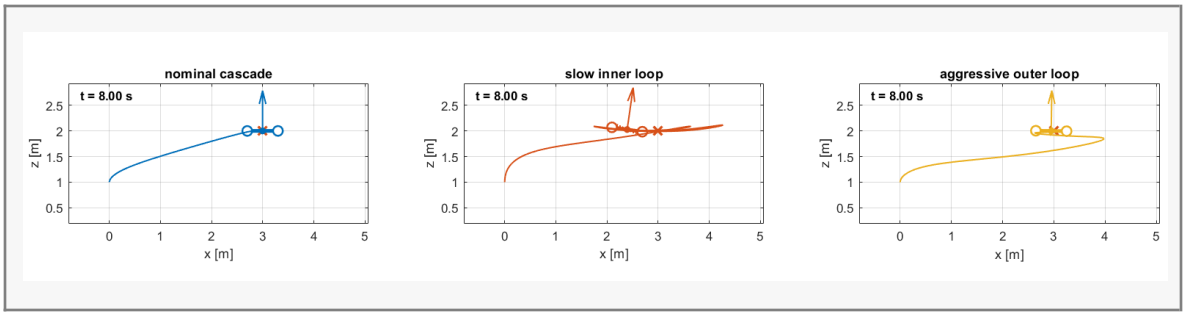
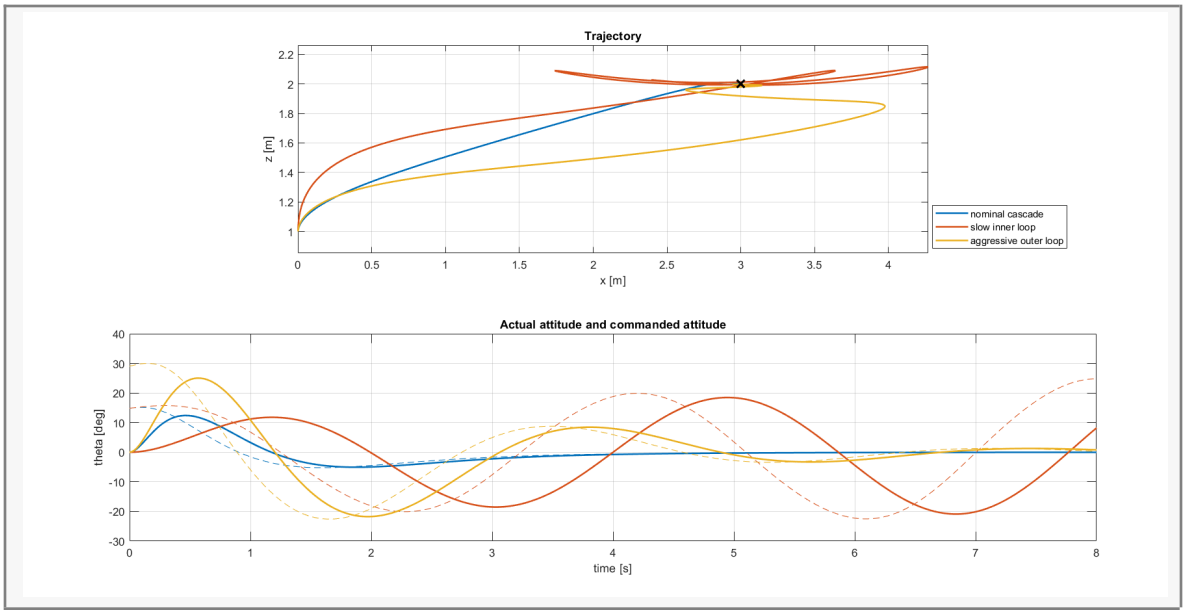


Figure 4: $\theta(t)$ and $\theta_d(t)$ comparison



Short Discussion

What to write: the most stable case, the most oscillatory case, the slow-inner-loop case, and the overly strong outer-loop case.

Entry field:

It seems that by inheriting the best tuning values I found in the previous two tasks we managed to successfully make a model that both quickly and accurately reaches the target position and attitude, which makes sense as the two previous tasks were about optimizing attitude control loop (or inner loop in a cascade control scenario) and a translation control loop respectively. Combining both into a cascade control loop. The other two previous cases do show that improperly balanced values causes a drone to be unable to stabilize due to either attitude correction being too weak, meaning it can't reach the desired attitude for both the target attitude and desired movement for translational correction, and what happens when the translational correction is too strong relative to the speed at which the quadrotor is capable of correcting its attitude.

Final Summary

Complete each item in 1–2 sentences.

Item	Entry Field
Reason the UAV moves laterally	Applied force is relative to quadrotor attitude (meaning it is "pushed" in the direction opposite to its top)
Role of the attitude loop	Correct quadrotor attitude to the desired value relative to the attitude difference and rotational inertia.
Role of the translation loop	As above, except for desired position in 3D space relative to the positional difference, inertia, and current attitude (as this dictates the direction of movement)
Condition for cascade control to work well	A proper balance of proportional gain / dampening and tracking based on the hardware, environmental conditions and desired application/movement/stability of the quadrotor. Attitude must be always taken in consideration and therefore takes priority to be computed and applied compared to translation.
What happens when the outer loop is too strong	Quadrotor will apply forces in too great an increment relative to how much it can correct its attitude for the purpose of reaching a desired position/attitude, resulting in great overshoots and larger need for correction.

Final Gain Table

Gain	Final selected value
p.att.kp	1.0
p.att.kd	0.25
p.pos.kp(1)	1.0

p.pos.kp(2)	1.5
p.pos.kd(1)	1.775
p.pos.kd(2)	2.0